Parallel Version of w03 Simulator II

Danilo Kaćanski

Faculty of Technical Sciences, Novi Sad
Applied algorithms in Control Systems

May 2026.

New Trace Model

Why did the Trace Model change?

- In the basic simulator, the runtime had one scheduler loop.
- Events were naturally ordered by:

`Step`

- In the concurrent version, many goroutines can produce events at the *same time*.
- That means the trace must handle:
  - concurrent writes
  - interleaved events
  - safe reads while execution is still active

Key Idea

The trace changed from a simple ordered step log into a thread-safe concurrent event log.

Event Model Change

Basic Event

```
type Event struct {
    Step int
    Type EventType
    Message *Message
    Process ProcessID
    Detail string
}
```

- step-based execution
- events belong to a scheduler turn

Concurrent Event

```
type Event struct {
    Seq int64
    Time time.Time
    Type EventType
    Message *Message
    Process ProcessID
    Detail string
}
```

- sequence-based logging
- events also store wall-clock time

Step vs Seq

Basic Version: `Step`

- tied to the runtime scheduler loop
- one step can contain multiple sub-events
- works because the runtime processes one message at a time

Concurrent Version: `Seq`

- global counter for logged events
- independent from a step loop
- gives a total order to events produced by multiple goroutines

Key Idea

In the basic model, the unit of execution is a **step**. In the concurrent model, the unit of observation is a **logged event**.

Why does the Trace need a Mutex?

- In the concurrent runtime, many goroutines may call: `trace.Log(...)` at the same time.
- Without synchronization, concurrent appends to the same slice are unsafe.

Concurrent trace state

```
type Trace struct {
    mu sync.Mutex
    events []Event
    verbose bool
    seq int64
}
```

Key Idea

The mutex protects the trace as shared state.

How Log(...) changes?

Basic Trace

- append event to slice
- optionally print it
- no locking needed

Concurrent Trace

- lock the mutex
- increment global `Seq`
- attach current `Time`
- append event safely
- unlock the mutex
- optionally print it

Why Add Timestamps?

- In the basic version, `Step` already explains where an event belongs.
- In the concurrent version, events happen in real time.
- `Time` gives extra information for debugging:
  - when the event happened
  - how far apart events were
  - whether activity stopped before idle timeout

Important

`Seq` gives event order.
`Time` gives wall-clock context.

Trace Comparison

Basic Trace

- `Step int`
- simple ordered log
- no mutex
- direct slice access
- one scheduler path
- deterministic step-based trace

Concurrent Trace

- `Seq int64 + Time`
- thread-safe event log
- protected by `sync.Mutex`
- snapshot reads
- many goroutines logging
- interleaved event trace

Link Layer Changes

What Stayed the Same?

Core link abstractions are unchanged

- **FairLossLink**
  - messages may be dropped
- **StubbornLink**
  - retransmits messages repeatedly
- **PerfectLink**
  - removes duplicates
- **AuthenticatedLink**
  - verifies MACs to detect tampering

Key Idea

The distributed systems theory behind the links is the same in both versions.

What Changed?

- Links are no longer controlled by one centralized step loop.
- Messages now move asynchronously.
- Retransmission is now periodic and asynchronous.
- Shared link state must now be thread-safe.
- Random loss generation is now owned by the links themselves.

Step-Based vs Concurrent Link Execution

Basic Version

- runtime calls: `Send(...)`
- runtime calls: `Tick(...)`
- runtime calls: `Receive(...)`
- all synchronized to: `Step()`

Concurrent Version

- processes send asynchronously
- router delivers through channels
- retransmitter runs in its own goroutine
- delivery is interleaved by concurrency

Key Idea

Links are no longer part of one scheduler turn.

Fair-Loss Link Change

Basic Fair-Loss Link

- runtime provides:

```
*rand.Rand
```

- randomness controlled externally
- safe because execution is single-threaded

Concurrent Fair-Loss Link

- link owns its own RNG
- RNG protected by:

```
sync.Mutex
```

- safe for concurrent access by many goroutines

Key Idea

The link becomes self-contained and concurrency-safe.

Retransmission Change

Basic Stubborn Link

- retransmission controlled by:

```
Tick(step)
```

- retransmission tied to scheduler steps
- runtime explicitly asks:
  "Should we retransmit now?"

Concurrent Stubborn Link

- runtime periodically calls:

```
Retransmissions()
```

- retransmitter goroutine injects retry traffic
- retransmissions behave like normal network traffic

Why do Links need Mutexes?

- In the concurrent runtime, links are shared between goroutines.
- Multiple goroutines may:
  - send messages
  - retransmit messages
  - verify authentication
  - check deduplication state
- Shared internal state must therefore be protected.

Examples

- `FairLossLink`: protects RNG
- `StubbornLink`: protects retransmission buffer
- `PerfectLink`: protects delivered-message set

Authentication Order Change

Basic Authenticated Link

1. deduplicate
2. verify MAC

Concurrent Authenticated Link

1. verify MAC
2. deduplicate

- retransmissions now happen asynchronously in separate goroutines
- tampered and valid retransmitted messages may arrive in unpredictable order
- if deduplication happens before MAC verification, an invalid message may occupy the deduplication state
- later, the valid retransmission may be rejected as a duplicate

Link Layer Comparison

Basic Links

- step-driven
- runtime-controlled RNG
- retransmission through `Tick()`
- no mutexes needed
- deterministic scheduling model

Concurrent Links

- asynchronous behavior
- self-owned RNG
- retransmission goroutine
- mutex-protected state
- interleaved concurrent execution

Examples in the Concurrent Runtime

Ping-Pong Example

Basic Version

- synchronous:

```
Handle(msg)
```

- returns replies directly
- runtime controls all scheduling
- passive state-machine style

Concurrent Version

- goroutine-based:

```
Run(ctx, inbox, send)
```

- waits on inbox channel
- emits replies asynchronously using `send(...)`
- process becomes an active participant

Counter Example

Basic Version

- plain integer state
- synchronous handlers
- runtime serializes all execution
- no synchronization needed

Concurrent Version

- reused using:

```
WrapHandler(...)
```

- counter uses:
  - `atomic.AddInt64(...)`
  - `atomic.LoadInt64(...)`
- state access must now be concurrency-safe

Tamper Example

Basic Version

- sender and receiver use:

```
Handle(msg)
```

- direct state fields:
  - received count
  - last received data

Concurrent Version

- rewritten as active goroutines
- uses:

```
Run(ctx, inbox, send)
```

- concurrency-safe storage:
  - `atomic.AddInt64(...)`
  - `atomic.Value`
- interceptor still simulates tampering

Overall Example Transition

```
Handle(msg) []Message
```

```
Run(ctx, inbox, send)
```

- processes become active goroutines
- messages arrive asynchronously through inbox channels
- outgoing communication happens through callbacks
- shared state must become concurrency-safe